

FEATURE REFERENCE · LIVING DOCUMENT

OxynDB

A complete, descriptive catalogue of every feature built into FoxByte — a PostgreSQL platform with instant branching, time-travel, and a database-per-AI-agent model. For each feature: **what it is**, **why we built it**, **what it does**, and **how it works**.

Product: FoxByte · **Type:** PostgreSQL with instant branching · **License:** AGPL-3.0 core / Apache-2.0 clients

Status: MVP complete (branching, time-travel, the gateway, HA, auth, web app)

This document is editable — the HTML source is designed to grow as new features ship.

Contents

Fifteen features across six areas. Each links to its own page.

FOUNDATION

01 **Performance-first hybrid architecture** Superpowers off the commit path

DATA & DURABILITY

02 **Instant copy-on-write branching** Full DB copies in seconds

03 **Time-travel & point-in-time recovery** Rewind to any moment

04 **Object storage & WAL archival** Durable, off-path backups

THE FRONT DOOR

05 **The Gateway — one smart endpoint** Route by database name

06 **Auto-suspend & auto-resume** Idle branches stop & wake

07 **Gateway authentication** API key as the DB password

AVAILABILITY & AI

08 **High availability & failover** Standby + transparent promote

09 **Agent Branch API** A database per AI agent

CONTROL & OPERATIONS

10 **Control-plane REST API** Manage & observe over JSON

11 **Background / daemon mode** One command, all services

ACCESS, SECURITY & EXPERIENCE

12 **Authentication & accounts** Password + OAuth + sessions

13 **API keys** Non-interactive credentials

14 **Web application & SQL console** Dashboard, guide, browser

15 **One-command cross-platform install** Install & run, no VM juggling

Performance-first hybrid architecture

The design principle underneath everything: give PostgreSQL superpowers without ever taxing an ordinary transaction.

WHAT IT IS

The architectural stance of the whole product. FoxByte runs **stock PostgreSQL on local NVMe** for the hot read/write path, and moves durability, branching, and time-travel **off** the commit path entirely. It is the postgres.ai / Database Lab model, implemented in Go.

WHY WE BUILT IT

The non-negotiable requirement was that **transaction speed is never compromised**. The popular approach separates compute from storage over the network, which adds latency to every commit and read. We deliberately rejected that trade-off: you should get instant branches and a rewind button *without* paying a tax on normal queries.

WHAT IT DOES

Everyday reads and writes hit a normal Postgres on a local disk at native latency — nothing special sits in front of them on the data path. Meanwhile, three "superpowers" are layered on using systems that live outside the commit path: instant branches, time-travel, and a single front door.

HOW IT WORKS

Each database is a normal `postgres:16` process with its data directory on a **ZFS** dataset on local storage. Write-ahead log (WAL) is shipped **asynchronously** to object storage, so archival never blocks a commit. Branches are created by **ZFS copy-on-write clones**, and the high-availability standby streams asynchronously. All the orchestration — routing, lifecycle, backups, branches — lives in a Go control layer beside Postgres, never in the query path.

Instant copy-on-write branching

A branch is an instant, isolated, full copy of the entire database — like `git branch`, but for live data.

WHAT IT IS

The signature feature. A **branch** is a complete, independent copy of the database that you can read and write freely, created in seconds and taking almost no disk — because it shares storage with its parent until you change something.

WHY WE BUILT IT

Developers and AI agents constantly need **production-like data** to experiment with — for a pull-request preview, a risky migration rehearsal, a CI run, or a per-agent sandbox. Physically copying a large database is slow and expensive, so people test against stale or tiny data. Branching removes that compromise: real, full-size data, instantly, disposable.

WHAT IT DOES

Creating a branch is **instant** and near-zero space; the branch is **fully isolated** (writes never touch the parent, and vice-versa); and you pay disk only for the **delta** you change. Branches can be listed, deleted, suspended, and resumed.

HOW IT WORKS

Under the hood a branch is a `zfs snapshot` of the parent dataset followed by a `zfs clone` — an O(1), copy-on-write operation regardless of database size. Each branch gets its own Postgres container mounting the clone as its data directory. Only blocks that diverge from the parent consume new space (shown as the "USED" copy-on-write delta).

Try it `fox branch create qa` · `fox branch list` · `fox branch delete qa`

Time-travel & point-in-time recovery

A rewind button for your data — restore the database to any moment within the archived window.

WHAT IT IS

Continuous backup with **point-in-time recovery (PITR)**: the ability to reconstruct the database exactly as it existed at a chosen timestamp (or "latest").

WHY WE BUILT IT

Mistakes happen — a bad migration, an accidental `DELETE`, a bug that corrupts data. Nightly dumps lose everything since the last snapshot. Time-travel lets you recover to the moment *before* the incident, and lets you inspect historical state for audits or debugging.

WHAT IT DOES

It continuously captures every change, so you can create a fresh, disposable restored instance at any past point without touching the live database. `fox backup create` takes a base backup; `fox restore --to '<timestamp>'` (or `--to latest`) materialises that state on a separate port.

HOW IT WORKS

The primary archives its write-ahead log (WAL) **continuously and asynchronously** to object storage using **wal-g**, alongside periodic base backups. A restore starts from the nearest base backup and **replays WAL** forward to the requested target time into a new container — the standard PostgreSQL PITR mechanism, automated end-to-end.

Try it `fox backup create` · `fox restore --to latest`

Object storage & WAL archival

The durable substrate that makes time-travel possible — without slowing down commits.

WHAT IT IS

An S3-compatible object store (MinIO in the dev stack) that holds the archived WAL segments and base backups the durability and recovery features depend on.

WHY WE BUILT IT

Durability has to live **off the commit path** to honour the performance principle, and it needs a cheap, scalable place to put a continuous stream of WAL. Object storage is the natural fit and mirrors how cloud databases achieve durability at scale.

WHAT IT DOES

Every WAL segment produced by the primary is shipped to the store shortly after it is written; base backups are stored there too. The restore path reads from the same bucket. Because archival is asynchronous, it never adds latency to a transaction.

HOW IT WORKS

A MinIO container exposes an S3 API and a console; a WAL bucket is created on startup. PostgreSQL's `archive_command` runs `wal-g wal-push` to upload each segment. All of this is provisioned automatically as part of bringing the stack up.

Try it [MinIO console at http://localhost:9001](http://localhost:9001)

The Gateway — one smart endpoint

A single PostgreSQL address that understands which branch you want and routes you there.

WHAT IT IS

The front door: one wire-protocol endpoint (`:6432`) that every client connects to, which routes each connection to the correct branch based on the database name in the connection string.

WHY WE BUILT IT

Branches come and go and each runs its own Postgres on its own port — clients should not have to track that. A stable, single address that "just knows" where to send you is what makes branching usable in real applications and tooling.

WHAT IT DOES

Connect to `...:6432/main` or `...:6432/feature-x` and the Gateway sends you to that branch. Your application uses one connection string forever; switching data is just changing the database name. It also transparently wakes suspended branches (feature 06) and enforces authentication (feature 07).

HOW IT WORKS

The Gateway is a Go TCP server that speaks the PostgreSQL wire protocol. It reads the client's **startup message**, extracts the requested database (the branch name), ensures that branch is running, dials its backend, and then pipes the session through transparently. Because it operates at the protocol level, ordinary drivers, ORMs, and tools work unchanged.

Try it `psql "postgres://dbadmin@localhost:6432/main"`

Auto-suspend & auto-resume

Idle branches stop themselves to free resources, and wake instantly on the next connection.

WHAT IT IS

The scale-to-zero behaviour: a branch that has been idle for long enough is automatically suspended, and is transparently resumed when someone connects to it again.

WHY WE BUILT IT

If every branch (per developer, per PR, per agent) ran forever, resources would balloon. Idle databases have to cost nothing — but suspension must be invisible, so users never think about it.

WHAT IT DOES

A branch idle beyond a configurable window (`--idle` , default 2 minutes) with zero active connections is stopped; its data is fully preserved. The next connection through the Gateway brings it back in a couple of seconds. Suspension can also be triggered or reversed manually.

HOW IT WORKS

A reaper goroutine inside the Gateway tracks the last time each branch was routed to and checks live server activity; when a branch is idle with no client backends it stops the container (data on ZFS survives). On the next connect, the router recreates or starts the container from its surviving dataset before forwarding. The primary, standby, and restore targets are deliberately excluded so availability is never affected.

Try it `fox gateway --idle 90s` · `fox branch suspend qa` · `fox branch resume qa`

Gateway authentication

The SQL endpoint requires a FoxByte API key as the password — the secret key finally gates database access.

WHAT IT IS

Authentication on the SQL front door. To connect through the Gateway you must present a valid FoxByte **API key** as the connection password; the real database password never leaves the Gateway.

WHY WE BUILT IT

Without it, anyone who could reach port `6432` could connect with a known password — the last open door in the platform. This closes it and unifies the security model: the same secret key that authorises the HTTP APIs now authorises database connections.

WHAT IT DOES

A client connects with its key as the password. An invalid key is rejected with a clear error; a valid key is admitted and routed to the requested branch. For trusted local use, authentication can be switched off with an environment flag.

HOW IT WORKS

After reading the startup message the Gateway sends `AuthenticationCleartextPassword`, reads the client's password, and validates it as an API key (hashed lookup). It then logs in to the backend Postgres **on the client's behalf** — performing the full handshake, including **SCRAM-SHA-256** — and finally sends its own `AuthenticationOk` to the client before piping the session. The client only ever sees its key; the backend credentials stay internal.

Try it `PGPASSWORD=key_... psql "postgres://dbadmin@localhost:6432/main"`

FEATURE 08

High availability & failover

A hot standby streams from the primary; a promotion reroutes clients without changing a single connection string.

WHAT IT IS

Replication and failover for the primary database: a continuously-updated standby that can be promoted to take over, with client traffic rerouted automatically.

WHY WE BUILT IT

A primary can fail. Applications should survive that without downtime or reconfiguration — ideally keeping the exact same connection string. This demonstrates the replication → promotion → rerouting mechanism end-to-end.

WHAT IT DOES

`fox ha enable` provisions a hot standby that streams changes from the primary with minimal lag; `fox ha status` shows replication state; `fox ha failover` promotes the standby and makes the Gateway route `main` to it, so clients keep working through the same endpoint. `fox ha disable` removes it.

HOW IT WORKS

The standby is built with `pg_basebackup` and streams WAL **asynchronously** (no commit-latency cost). Failover calls `pg_promote()` and updates a primary-pointer file that the Gateway consults, so a connection to `main` transparently lands on the promoted node. The auto-suspend reaper is careful never to stop the standby.

Try it `fox ha enable` · `fox ha status` · `fox ha failover`

Engine: streaming replication + `pg_promote()`

Agent Branch API

One instant, isolated, disposable database per AI agent — over a simple HTTP call.

WHAT IT IS

FoxByte's AI-native interface: an HTTP API that hands each AI agent its own private database branch, created and destroyed on demand.

WHY WE BUILT IT

Autonomous agents need somewhere to read and write state safely — without corrupting shared data or interfering with each other. Manually provisioning databases per agent is impractical. This turns "a database per agent" into a single API call, which is the product's long-term north star (an agent state substrate).

WHAT IT DOES

`POST /agents/{id}/branch` returns a ready-to-use connection string for a fresh, isolated database; the agent uses it like any PostgreSQL; `DELETE /agents/{id}/branch` tears it down. Each agent's data is invisible to the others and to production.

HOW IT WORKS

The API (on `:8088`) wraps the branching engine (feature 02): a request creates a copy-on-write branch named for the agent and returns its DSN. The endpoints are authenticated with an API key, so only authorised callers can mint agent databases.

Try it `curl -H "Authorization: Bearer key_..." -X POST localhost:8088/agents/alice/branch`

FEATURE 10

Control-plane REST API

A JSON API to manage and observe the whole platform — and the engine behind the web app.

WHAT IT IS

A REST/JSON management API (on `:8080`) that exposes the platform's state and operations programmatically.

WHY WE BUILT IT

Operations shouldn't require shell access to the host. A clean API makes FoxByte scriptable and automatable, and provides a single backend that the web UI and any external tooling can share.

WHAT IT DOES

It reports live status (primary readiness, branch and agent counts, HA state, storage), lists and creates branches, suspends/resumes and deletes them, and runs SQL against any branch — all over authenticated HTTP.

HOW IT WORKS

Built on Go's standard `net/http` with method-and-path routing; SQL execution uses the **pgx** driver against the target branch. Every `/api/*` route is wrapped in the authentication middleware and served with credentialed CORS so the browser app can call it securely.

```
Try it curl -H "Authorization: Bearer key_..." localhost:8080/api/status
```

Engine: internal/controlplane (net/http + pgx)

FEATURE 11

Background / daemon mode

One command brings the entire platform up in the background; stop, status, and logs manage it.

WHAT IT IS

Process management for the long-lived services (Gateway, control plane, agent API) so the platform runs detached rather than tying up terminals.

WHY WE BUILT IT

A database platform must run unattended. Requiring several foreground terminals to keep servers alive is unworkable for real use, demos, or self-hosting.

WHAT IT DOES

`fox start` brings up the stack and all servers in the background; `fox stop` tears everything down; `fox status` shows what's running; `fox logs [gateway|api]` prints a server's log.

HOW IT WORKS

Each service is launched as a detached process (via `setsid`) with its PID and log file recorded under `~/.fox/`. A small services map defines what `start` / `stop` manage, and initialisation is idempotent so repeated starts are safe.

Try it `fox start` · `fox status` · `fox logs gateway` · `fox stop`

Engine: internal/daemon

Authentication & accounts

User accounts with passwords, optional social sign-in, and secure sessions — the platform's security foundation.

WHAT IT IS

The identity layer: email/password accounts, optional GitHub and Google sign-in, and server-side sessions that gate every protected surface.

WHY WE BUILT IT

To share or expose FoxByte beyond a single trusted machine, access must be controlled. Authentication secures the dashboard, console, and APIs, and is the groundwork for future multi-tenant "projects".

WHAT IT DOES

Users can register, log in, and log out; a secure session cookie keeps them signed in; social login is offered when configured. Protected pages and API routes reject unauthenticated requests.

HOW IT WORKS

Account data lives in a self-contained **SQLite** database (`~/ . fox/auth.db`). Passwords are stored as **bcrypt** hashes (never plaintext). Sessions are opaque random tokens kept in an **httpOnly** cookie and validated server-side. OAuth uses the standard authorization-code flow with GitHub/Google, enabled only when client credentials are supplied.

Try it `fox user create you@example.com` · or sign up in the web app

FEATURE 13

API keys

Long-lived credentials for machines, agents, and CI — and the password that unlocks the Gateway.

WHAT IT IS

Named, revocable API keys tied to a user account, used for non-interactive access to the HTTP APIs and as the password for SQL connections through the Gateway.

WHY WE BUILT IT

Automations, agents, and CI can't log in interactively — they need a stable secret. Keys also give the Gateway (feature 07) a credential to check, unifying HTTP and SQL access under one identity model.

WHAT IT DOES

You can create keys (shown once), list them by prefix, and revoke them instantly. A key is presented as a `Bearer` token to the APIs, or as the password when connecting through the Gateway; revoking it cuts off everything using it.

HOW IT WORKS

Each key is a random secret with a `key_` prefix. Only a **SHA-256 hash** plus a short display prefix is stored — the full key is never recoverable. Verification hashes the presented key and looks it up, so a leaked database never exposes usable credentials.

Try it `fox apikey create you@example.com` · or the [API keys page](#) in the web app

FEATURE 14

Web application & SQL console

A polished way to learn, operate, and explore the database — landing, guide, docs, dashboard, and a pgweb-style console.

WHAT IT IS

A standalone web app that presents FoxByte: a marketing landing page, a developer guide, a reference, a live operations **dashboard**, and a **SQL console / schema browser**.

WHY WE BUILT IT

A great database platform needs a great front end — for onboarding, day-to-day operations, and data exploration — without dropping to the terminal. It also showcases the product and provides a consistent, branded experience.

WHAT IT DOES

The **dashboard** shows live status and lets you create, suspend, resume, and delete branches and copy their connection strings. The **console** browses a branch's tables and views and, for any table, shows its **Rows**, **Structure**, and **Indexes** — or runs arbitrary SQL. The **guide** and **docs** teach and reference the product. Authenticated pages are protected by sign-in.

HOW IT WORKS

It is a **Vite + React + TypeScript** single-page app that consumes the control-plane REST API — it is not embedded in the database binary. A theme-aware design system provides light/dark modes; an auth context guards protected routes; and the schema browser is built entirely on the API's SQL endpoint.

Try it `make web-dev` → `http://localhost:5173`

Engine: web/ (React) over the control-plane API

One-command cross-platform install

Install the `fox` command and run one command — the Linux VM, Docker, ZFS, pool, and image are set up for you.

WHAT IT IS

The distribution story: a single installer and a single command bring FoxByte up on macOS and Linux (with Windows as a fast-follow), hiding all the infrastructure setup.

WHY WE BUILT IT

The engine needs Linux + Docker + ZFS, which historically meant a long, error-prone setup. For anyone to actually adopt FoxByte, getting started had to collapse to "install, then run."

WHAT IT DOES

A one-line installer places the `fox` command; then `fox setup` (macOS) or `fox start` (Linux) creates the environment, provisions the copy-on-write pool and the image, and brings the database, Gateway, and APIs up — no manual steps. Every day-to-day command is just `fox ...` on every OS.

HOW IT WORKS

The `fox` binary is a **launcher**: on macOS it transparently manages and forwards commands into a Lima VM; on Linux it runs natively. The engine's `Provision()` step auto-creates the ZFS pool (on a loopback file), datasets, and Docker image on first run. Release binaries for all platforms are produced by a cross-compilation step, and the installer fetches the right one.

Try it `curl -fsSL .../install.sh | sh` · `fox setup`

Engine: internal/host launcher + install.sh + make release